

Multicycle CPU impliment

2024084648 이종민

1. Implement

1) GLOBAL.v

```
13
14+ `define IF 3'b000
15+ `define ID 3'b001
16+ `define EX 3'b010
17+ `define MEM 3'b011
18+ `define WB 3'b100
```

State 상태 매크로 IF, ID, EX, MEM, WB 정의

2) CTRL.v

```
module CTRL(
    // input opcode and funct
    input rst,
    input clk,
    input [5:0] opcode,
    input [5:0] funct,

    // output various ports
    output reg [1:0] PCSource,
    output reg [3:0] ALUOp,
    output reg [2:0] ALUSrcB,
    output reg ALUSrcA,
    output reg RegWrite,
    output reg [1:0] RegDst,

    output reg PCWriteCond,
    output reg PCWrite,
    output reg IorD,
    output reg MemRead,
    output reg MemWrite,
    output reg MemtoReg,
    output reg IRWrite,

    output reg SignExtend
);
```

클럭마다 state를 조정하기 위해 과제 명세 외에 추가로 rst, clk를 input으로 받음.

```
reg [2:0] state;
reg [2:0] state_next;
reg InstDone;
```

State를 위한 state, state_next reg와 한 명령어가 끝났음을 나타내는 InstDone 정의

```
//clk마다 state 바꾸기
always @(posedge clk) begin
    if (rst) state <= `IF;
    else if (InstDone) state <= `IF;
    else state <= state_next;
end
```

Positive edge clock마다 reset이면 state를 IF, InstDone이면 IF, 이외에는 state_next로 state변경함

```

always @(*) begin
    //initialize to 0
    PCSrc = 2'b00; ALUOp = 4'b0000; ALUSrcB = 2'b00;
    ALUSrcA = 0; RegWrite = 0; RegDst = 0;
    PCWriteCond = 0; PCWrite = 0; IorD = 0;
    MemRead = 0; MemWrite = 0; MemtoReg = 0;
    IRWrite = 0; SignExtend = 1; InstDone = 0;
    state_next = state;

    case (state)
    //Instruction Fetch
    `IF: begin
        end

    //Instruction Decode
    `ID: begin
        //R-Type instruction
        //I-Type instruction
        //J-Type instruction
        end

    //Execution
    `EX: begin
        //I-Type instruction
        end

    //Memory
    `MEM: begin
        //I-Type instruction
        end

    //Write Back
    `WB: begin
        //I-Type instruction
        //J-Type instruction
        end
    endcase
end

```

우선 Reg값들을 전부 초기화 한다.

이후 case 문을 이용해 state에 따라 instruction마다 알맞은 컨트롤 신호를 보내주고 다음 state를 정해준다.

만약 instruction이 끝나는 상태면 InstDone을 1로 한다.

3) CPU.v

```

// Control-related wires
wire [1:0] PCSrc;
wire [3:0] ALUOp;
wire [2:0] ALUSrcB;
wire ALUSrcA;
wire RegWrite;
wire [1:0] RegDst;
wire PCWriteCond;
wire PCWrite;
wire IorD;
wire MemRead;
wire MemWrite;
wire MemtoReg;
wire IRWrite;
wire InstDone;
wire SignExtend;

```

Ctrl을 위한 wire를 정의한다.

```

// RF-related wires
wire [4:0] rd_addr1;
wire [4:0] rd_addr2;
wire [31:0] rd_data1;
wire [31:0] rd_data2;
wire [31:0] AWire;
wire [31:0] BWire;
wire [4:0] wr_addr;
wire [31:0] wr_data;

// ALU-related wires
wire [31:0] operand1;
wire [31:0] operand2;
wire [31:0] alu_result;
wire zero;
reg [31:0] ALUOut;
wire [31:0] ALUOutWire;

//A,B reg
reg [31:0] A;
reg [31:0] B;

// MEM-related wires
wire [31:0] mem_addr;
wire [31:0] mem_write_data;
wire [31:0] mem_read_data;

//memory data register
reg [31:0] MDR;
wire [31:0] MDRWire;

```

Microarchitecture를 위한 A, B, MDR, ALUOut과 그 각각의 와이어를 정의한다.

```

//새로 선언한 레지스터 와이어 연결
assign MDRWire = MDR;
assign AWire = A;
assign BWire = B;
assign ALUOutWire = ALUOut;

```

그 레지스터와 와이어를 연결한다.

```

//ALU wire 연결
assign operand1 = ALUSrcA ? AWire : PC;
assign operand2 = (ALUSrcB == 2'b00) ? B :
                  (ALUSrcB == 2'b01) ? 4 :
                  (ALUSrcB == 2'b10) ? ext_imm :
                  (ALUSrcB == 2'b11) ? (ext_imm << 2) :
                  0;

//MEM wire 연결
assign mem_addr = IorD ? ALUOut : PC;
assign mem_write_data = BWire;

//RF wire 연결
assign rd_addr1 = rs;
assign rd_addr2 = rt;
assign wr_addr = (RegDst == 2'b01) ? rd :
                (RegDst == 2'b00) ? rt :
                5'd31;
assign wr_data = MemtoReg ? MDRWire : ALUOutWire;

always @(*) begin
    if (PCSource == 2'b00)
        PC_next = alu_result;
    else if (PCSource == 2'b01)
        PC_next = ALUOutWire;
    else if (PCSource == 2'b10)
        PC_next = {PC[31:28], immj, 2'b00};
    else if (PCSource == 2'b11)
        PC_next = AWire;
end
end

```

ALUSrcA,B, 그리고 PCSource의 값에 맞게 oprand1,2와 PC_next를 mux로 연결하고
IorD, RegDst, MemtoReg에 맞게 mux로 연결한다.

```

// Update the Clock
always @(posedge clk) begin
    if (rst) PC <= 0;
    else begin
        if (PCWrite || (PCWriteCond && zero))
            PC <= PC_next;
        if (IRWrite)
            IR <= mem_read_data;
        if (MemRead)
            MDR <= mem_read_data;
        ALUOut <= alu_result;
        A <= rd_data1;
        B <= rd_data2;
    end
end
end

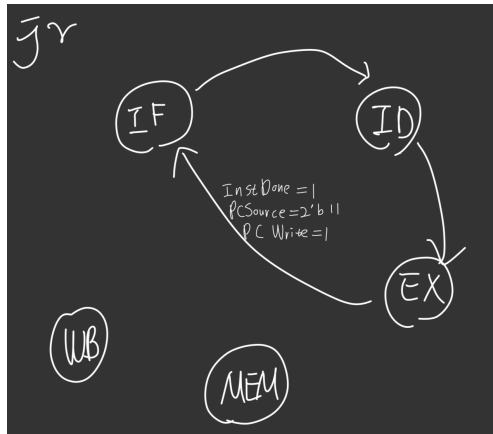
```

Positive edge clock마다 rst를 확인하고, 과제 명세와 같이 PC회로를 연결하고 한 클럭 전 상태를 저장하기 위해 IR, MDR, ALUOut, A, B를 클럭마다 업데이트 하는데,IR과 MDR 레지스터를 컨트롤 신호가 1이면 업데이트한다.

2. jr, jal FSM & Implimentation and μarchitecture

1) jr

FSM



```

always @(*) begin
    if (PCSrc == 2'b00)
        PC_next = alu_result;
    else if (PCSrc == 2'b01)
        PC_next = ALUOutWire;
    else if (PCSrc == 2'b10)
        PC_next = {PC[31:28], immj, 2'b00};
    else if (PCSrc == 2'b11)
        PC_next = AWire;
end
  
```

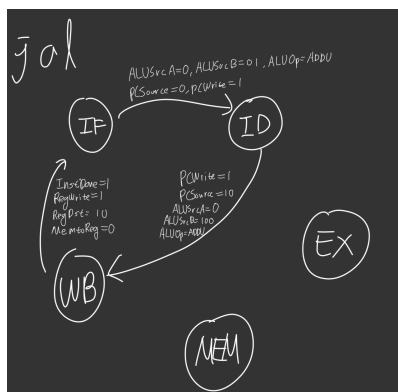
Implementation and μarchitecture

PCSrc를 A에서 받아와야 하지만 현재 PCSrc에서 그럴 수 없기에

PCSrc = 11 마이크로 아키텍처를 새로 정의하여 Awire에서 PC_next를 가져올 수 있도록 하였다.

2) Jal

FSM



```

assign operand2 = (ALUSrcB == 2'b00) ? B :
                  (ALUSrcB == 2'b01) ? 4 :
                  (ALUSrcB == 2'b10) ? ext_imm :
                  (ALUSrcB == 2'b11) ? (ext_imm << 2) :
                  0;
  
```

Implimentation and μarchitecture

IF stage에서 $PC = PC + 4$ 를 미리 실행하기 때문에 WB stage에선 그 값을 그대로 레지스터에 저장하고 싶지만 과제의 구조에선 ALUOut 값이나 MDR 값만 레지스터 파일의 wr_data로 사용 가능하다. 따라서 $ALUSrcB = 100$ 신호일 때 $operand2 = 0$ 이 되게 하여 $PC + 4 + 0$ 이 되어 알맞은 link 값을 r31에 저장할 수 있게 하였다.